

Technical Requirements Document

Visual Reasoning Question Engine — Paper Folding/Punching, Mirror & Rotation Question Generation

Field	Value
Document status	Draft v1.0
Date	18 July 2026
Prepared for	Aptitude / pilot-selection reasoning question generation (SSC, UPSC, DRDO, CAT style)
Owner	Lakshay Sachdeva
Related doc	"Visual Reasoning Engine — Architecture & Dataset Plan" (prior deliverable)

1. Purpose & Scope

1.1 Purpose

Define the functional and non-functional requirements, data contracts, and technology choices for a system that procedurally generates paper-folding/punching, mirror-image, and rotation/series reasoning questions with mathematically guaranteed-correct answers, verifies them with a vision-language model, writes exam-style question text with an LLM, and serves them through a filtered retrieval API.

1.2 In Scope

- Procedural generation of three question families: paper folding & punching, mirror/water image, rotation & series.
- Automatic answer + distractor computation (no human authoring required for correctness).
- Vision-model verification loop and scanned-book ingestion path.
- LLM-based question text, options, and explanation generation.
- Dataset storage, embedding index, and a retrieval API for pulling filtered sets of questions on demand.

1.3 Out of Scope (v1)

- Reasoning families beyond the three listed (e.g. verbal analogy, series completion in numbers/letters) — noted as a future extension in Section 12.
- End-user exam-taking UI / proctoring — this TRD covers the generation and retrieval backend only.
- Fine-tuning a dedicated model on the dataset — the dataset this system produces is an input to that, not covered here.

1.4 Glossary

Term	Meaning
Geometry engine	Deterministic code that computes fold/mirror/rotation transforms and the resulting correct answer.
VLM	Vision-language model, used to independently verify a rendered question.
Distractor	An incorrect answer option, generated by deliberately breaking one rule of the correct transform.

Term	Meaning
Record	One complete question entry: params, images, text, correct answer, verification status.
Retrieval API	Service that filters/searches the verified dataset and returns a requested set of questions.

2. System Overview

2.1 Problem Statement

Reasoning questions of these types (fold+punch, mirror, rotation) require a geometrically correct answer, which general-purpose LLMs and retrieval-augmented generation cannot reliably produce because the answer is a spatial computation, not information that exists in text form anywhere. The system must generate unlimited, correct, difficulty-tunable questions at scale, and make an existing verified set retrievable by topic/difficulty on request.

2.2 Solution Approach

Separate the system into three responsibilities that are each solved with the right tool: a deterministic geometry engine guarantees correctness; a vision-language model verifies each rendered item as an independent check and can also digitize existing scanned questions; an LLM converts already-correct structured data into exam-style text. A retrieval layer (embedding + metadata filter) sits on top of the verified dataset to serve filtered requests instantly, without regenerating anything.

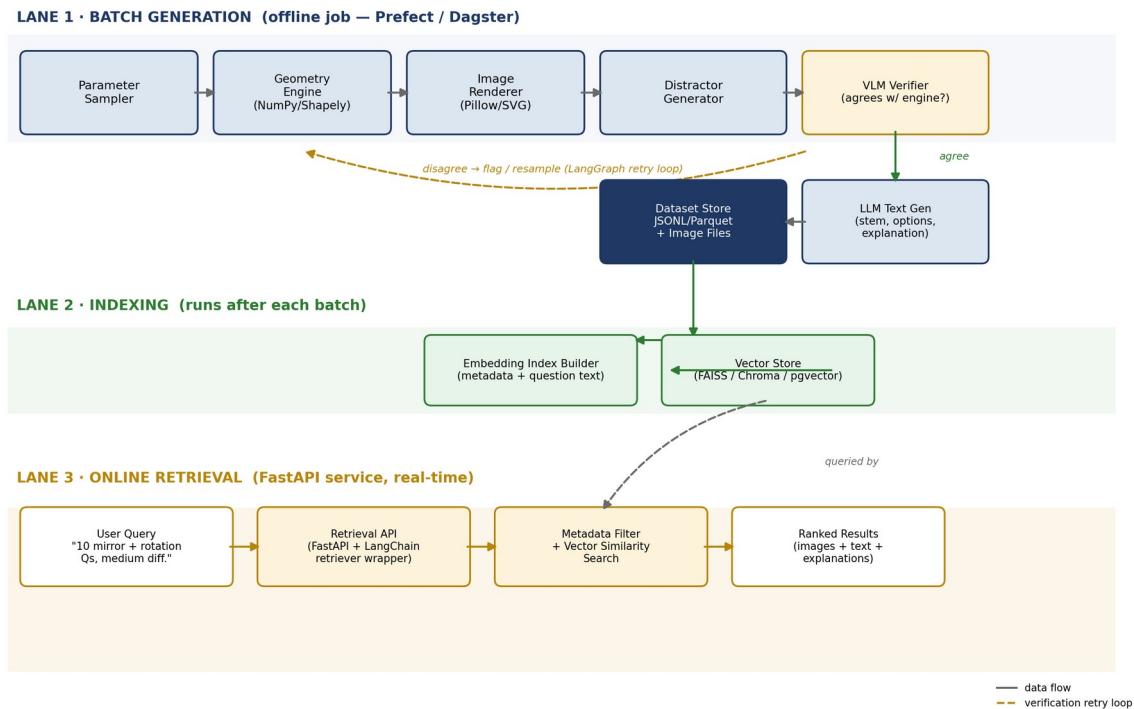
2.3 High-Level Component Map

Layer	Responsibility	Correctness guarantee
Geometry Engine	Compute fold/mirror/rotation transforms and correct answer + distractors	By construction (pure math)
Vision-Language Model	Independently solve each rendered question; digitize scanned questions	Statistical — used as a check, not a source of truth
LLM	Write stem, options, explanation from structured data	N/A — text only, never alters the answer
Retrieval API	Filter + rank the verified dataset for a request	Deterministic filter over pre-verified records

3. Pipeline Flow

Three lanes: an offline batch generation lane that produces and verifies questions, an indexing lane that builds the searchable index after each batch, and an online retrieval lane that serves filtered requests in real time.

Visual Reasoning Question Engine — Pipeline Flow



3.1 Lane 1 — Batch Generation (offline)

1. Parameter Sampler draws a random valid configuration (fold type/count, punch positions, mirror axis, rotation rule) from the parameter space, biased toward under-represented difficulty buckets.
2. Geometry Engine computes the transform and the exact correct-answer geometry.
3. Image Renderer draws the question image and the correct-answer image.
4. Distractor Generator produces 3 wrong options, each by breaking one specific rule, and tags which rule was broken.
5. VLM Verifier independently solves the rendered question. If it agrees with the engine's answer, the record proceeds. If it disagrees, the record is flagged and routed back to the sampler (retry with adjusted parameters) or to a human review queue if it fails repeatedly.
6. LLM Text Generator writes the stem, option labels, and step-by-step explanation from the structured record (distractor tags become the explanation of each wrong option).
7. The completed record is written to the Dataset Store.

3.2 Lane 2 — Indexing

After each batch, an Embedding Index Builder embeds each record's question text + tags (not image pixels) and writes it to a vector store, alongside the metadata fields used for filtering (family, sub_type, difficulty, exam_style).

3.3 Lane 3 — Online Retrieval

A FastAPI service accepts a request such as "10 mirror and rotation questions, medium difficulty," applies a metadata filter (family, difficulty, exam_style) combined with a vector similarity search (to diversify results and avoid repeats for a given user/session), and returns ranked results with images, text, and explanations. No generation happens in this lane — everything returned was already verified in Lane 1.

4. Functional Requirements

ID	Requirement	Priority
FR-1	System shall generate fold & punch questions for vertical, horizontal, and diagonal folds, 1–3 folds, 1–4 punches.	Must
FR-2	System shall generate mirror/water-image questions across vertical, horizontal, and arbitrary-angle axes for a configurable figure library.	Must
FR-3	System shall generate rotation/series questions with configurable rotation step size and sequence length.	Must
FR-4	For every generated question, the system shall compute the correct answer deterministically from the same parameters used to render the question (no independent/manual answer entry).	Must
FR-5	System shall generate exactly 3 distractors per question, each tagged with the specific rule it violates.	Must
FR-6	System shall verify every generated question with a VLM before it is marked verified in the dataset.	Must
FR-7	System shall support ingesting scanned question images from existing books/papers via the same VLM verification path, tagged source=scanned_book.	Should
FR-8	System shall generate question stem, option labels, and explanation text via an LLM from the structured record.	Must
FR-9	System shall expose a retrieval endpoint returning N questions filtered by family, difficulty, and exam style.	Must
FR-10	System shall avoid returning near-duplicate questions (by embedding similarity) within a single retrieval response.	Should
FR-11	System shall support difficulty tagging (1–5) derived from generation parameters (fold count, rotation steps, distractor subtlety).	Must
FR-12	System shall persist generation parameters for every record so any question can be re-rendered at a different resolution without recomputation.	Should

5. Data Requirements

One record per question, whether generated or digitized. Images stored as files; everything else as JSONL or Parquet.

Field	Type	Notes
question_id	UUID	Stable identifier
family	enum	fold_punch mirror_water rotation_series
sub_type	string	e.g. vertical_fold_double_punch, 45deg_mirror
difficulty	int 1–5	Derived from generation params
params	JSON	Raw generator parameters — enables re-render
image_paths	object	Question image + each option image
correct_option	string	
distractor_rules	list	Rule violated per wrong option
exam_style	enum	SSC UPSC CAT DRDO_pilot generic
text	object	stem, option_labels, explanation

Field	Type	Notes
tags	list	Free-form topic tags
embedding_text	string	Text used to build the vector index
verification	object	vlm_model, vlm_answer, agree, verified_at
source	enum	synthetic scanned_book
created_at / version	timestamp / int	Auditability, re-generation tracking

6. Non-Functional Requirements

Category	Requirement
Correctness	100% of shipped questions must have a geometrically-derived correct answer; VLM agreement rate on shipped questions $\geq 98\%$ (disagreements are auto-flagged, not shipped).
Throughput	Batch generation shall sustain at least 500 verified questions/hour per worker on commodity CPU (geometry + render only; VLM/LLM calls are the bottleneck and should be batched/parallelized).
Retrieval latency	Retrieval API p95 response time under 300 ms for requests returning up to 20 questions.
Scalability	Dataset store and vector index must support at least 200,000 records per family without redesign; batch workers must scale horizontally.
Reliability	Failed verifications retry up to 3 times with re-sampled parameters before routing to a human review queue; no silent drops.
Storage efficiency	Question/option images stored as compressed PNG or SVG; target under 50 KB average per image.
Observability	Track and expose: generation rate, VLM agreement rate, dedup rate, per-family/difficulty distribution, retrieval latency.
Auditability	Every record traceable to the exact parameters and model versions (geometry engine version, VLM model, LLM model) used to produce it.
Data sensitivity	Given the DRDO/pilot-selection context, access to the generation and retrieval APIs should be restricted (auth-gated); no personal candidate data is stored in this system — it only stores question content.

7. Technology Stack & Framework Decisions

Framework choice follows from where each component's job actually sits — deterministic computation, stateful multi-step control flow, or simple retrieval — rather than defaulting to one agent framework everywhere.

Component	Choice	Rationale
Geometry & transforms	Plain Python — NumPy / Shapely	Pure deterministic math. No LLM or agent framework involved — a framework here would only add risk, not value.
Rendering	Pillow / OpenCV or svgwrite	Straightforward 2D drawing from computed coordinates.
Generate → verify → repair	LangGraph	This step is a stateful loop with a real

Component	Choice	Rationale
control flow		conditional branch (VLM agrees → proceed; disagrees → resample/flag). LangGraph models exactly this as a graph with conditional edges, instead of forcing a linear chain to fake a loop.
Retrieval layer	LangChain retriever wrapper over FAISS / Chroma / pgvector (optional) or the vector DB SDK directly	LangChain saves boilerplate for the metadata + similarity filter combo, but is not required — a direct SDK call is equally valid since the query pattern is simple filtered search, not conversational RAG.
Batch job orchestration (scaling generation to 100k+ records)	Prefect or Dagster	This is a data-pipeline scheduling problem (retries, parallel workers, backfills), which workflow orchestrators handle natively; agent frameworks are not built for this.
Serving layer	FastAPI	Exposes /generate (internal/admin), /verify, and /questions (public retrieval) as REST endpoints.
Vision verification model	Swappable behind one interface — Qwen3-VL, GLM-4.5V, DeepSeek-VL (self-hosted) or Gemini 2.5 Pro (API)	Re-benchmark periodically against a held-out hand-checked set; don't hard-code one model.
Text generation model	Swappable behind one interface — current Claude / GPT / Qwen3 text models	Same reasoning — this is a config value, not an architectural decision.
Dataset storage	Images on disk / S3-compatible object storage; records in JSONL or Parquet	Parquet enables fast columnar filtering at scale for the retrieval layer.

Explicitly not used: a general-purpose agent framework (e.g. an autonomous LangChain agent) to "figure out" the geometry at generation time. That would reintroduce exactly the reliability problem this design avoids — the geometry engine must remain deterministic code, not an LLM decision.

8. API Contracts

8.1 POST /v1/generate (internal/admin)

Triggers a batch generation run.

Field	Example
Request	{ "family": "fold_punch", "count": 5000, "difficulty_range": [1,5] }
Response	{ "batch_id": "b-2026-0718-01", "status": "queued" }

8.2 GET /v1/questions (public retrieval)

Field	Example
Request (query params)	family=mirror_water,rotation_series&difficulty=2-3&exam_style=SSC&count=10

Field	Example
Response	<pre>{ "results": [{ "question_id": "...", "family": "mirror_water", "image_url": "...", "options": [...], "correct_option": "C", "explanation": "..." }, ... 10 items] }</pre>

8.3 GET /v1/questions/{id}

Returns a single full record, including params and verification metadata (for admin/debug use).

9. Testing & Validation Plan

Test type	What it covers
Unit tests — geometry engine	Fixed input params against hand-verified expected coordinates for each fold/mirror/rotation case; run on every code change.
Golden-set regression	A frozen set of ~200 known-correct records re-rendered and re-verified on every pipeline change to catch silent regressions.
VLM agreement monitoring	Track agreement rate per batch; alert if it drops below the 98% threshold — signals a rendering or distractor bug, not just a bad batch.
Human spot-check	Random sample (e.g. 1%) of newly verified records reviewed by a person before wide release, especially early on.
Retrieval load testing	Simulate concurrent filtered-retrieval requests to confirm the p95 latency target under Section 6.
Dedup validation	Confirm embedding-similarity dedup catches deliberately near-duplicate synthetic test records before they'd be indexed.

10. Risks & Mitigations

Risk	Mitigation
VLM false-agreement (verifier wrong in the same way the renderer is buggy)	Periodically cross-check a sample against a second, different VLM and against human review.
Distractors accidentally also correct (ambiguous question)	Golden-set regression + human spot-check catch this before wide release.
Dataset skew toward easy/common parameter combinations	Difficulty-bucket-aware sampling (Section 3.1, step 1) actively biases toward under-represented buckets.
Model drift/deprecation (VLM or LLM provider changes)	Both kept behind a single swappable interface (Section 7); re-benchmark on a fixed held-out set before switching.
Retrieval returning repetitive questions to the same user	Vector similarity diversification + per-user recently-served exclusion list.

11. Phased Delivery

Phase	Deliverable	Rough duration
1	Fold & punch generator end-to-end (engine, renderer, distractors, VLM verification loop as a LangGraph flow, schema, ~500 verified records).	2–3 weeks

Phase	Deliverable	Rough duration
2	Add mirror/water and rotation/series generators on the shared transform library; unify sampler across all three families.	2 weeks
3	Scale to 100k+ records with dedup and difficulty balancing under Prefect/Dagster; begin scanned-book ingestion.	3–4 weeks
4	Build embedding index + FastAPI retrieval endpoint; integrate with the consuming application.	1–2 weeks

12. Future Extensions

- Additional reasoning families (verbal analogy, number/letter series, embedded figures) on the same record schema.
- Adaptive difficulty selection in the retrieval API based on a candidate's prior performance.
- Fine-tuning a dedicated small VLM on the verified dataset once volume is sufficient, to reduce reliance on third-party model APIs.

Appendix A — Open Questions

- Target total dataset size per family and per difficulty tier?
- Self-hosted vs API-based VLM/LLM — depends on data-sensitivity requirements for a DRDO-context deployment.
- Which exam styles (SSC/UPSC/CAT/DRDO-pilot) are priority for the first LLM phrasing pass?